Progress Partners

DevOps Engineer

Take-Home-Assessment

TASK 05 - Architecture Design & ADR

Dilakshan Chandrasena

1. Executive Summary

This report proposes a practical AWS architecture for gradually migrating the existing monolithic application to microservices. The main goals are to support 10,000 requests per second, protect sensitive data, and ensure that network access follows the principle of least privilege.

The proposed solution uses Amazon ECS with AWS Fargate to run independently scalable services within a secure, multi-Availability Zone VPC. The migration will follow the Strangler Fig pattern, allowing services to be moved gradually while reducing disruption to existing operations.

The design also covers encryption, monitoring, load testing, data consistency, and rollback procedures to support a secure and reliable migration.

2. Requirements and Assumptions

2.1 Requirements

The proposed architecture must address the following requirements from the assessment brief:

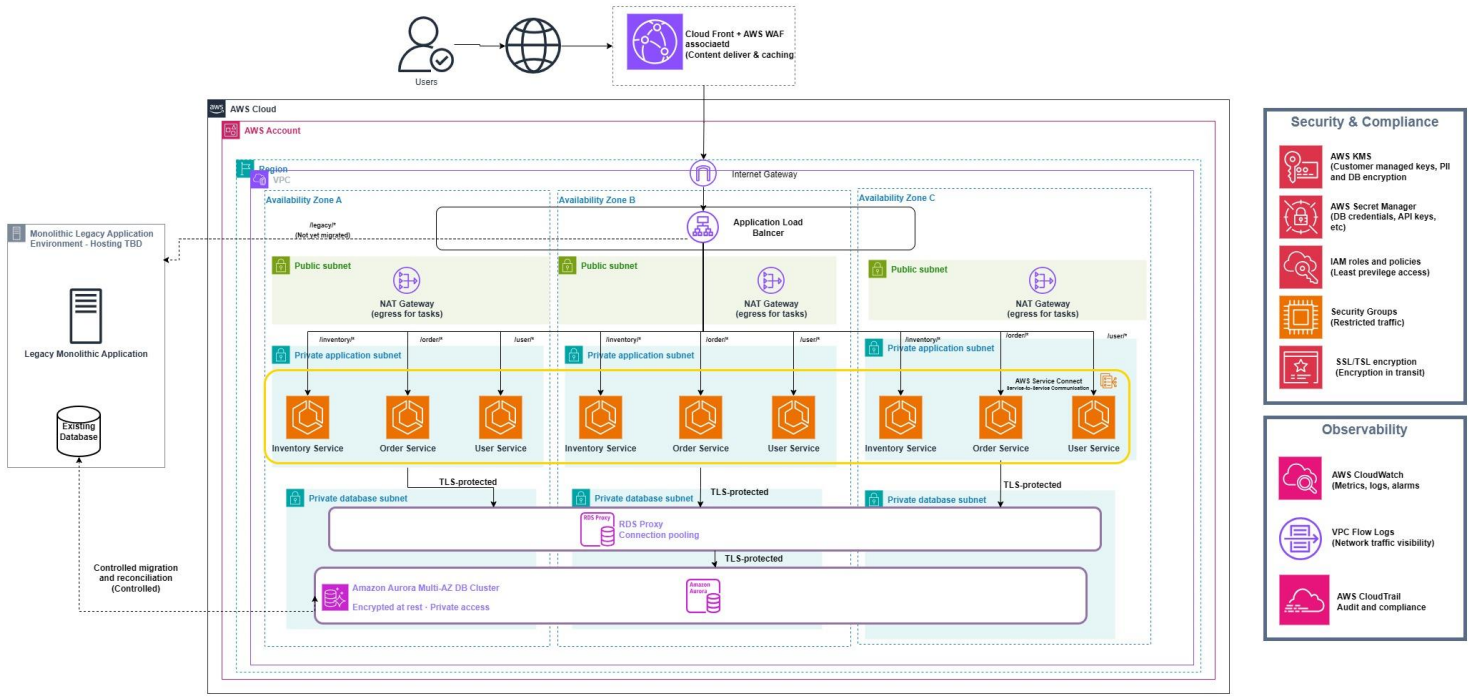
- **Scalability:** Support a target workload of 10,000 requests per second.
- **Data protection:** Encrypt personally identifiable information (PII) both at rest and in transit.
- **Network security:** Enforce least-privilege network access, allowing only required communication between components.
- **Application migration:** Support the transition from the existing monolithic application to a microservices architecture while minimising disruption.

2.2 Assumptions

As the assessment brief does not specify all production requirements, the following assumptions are used for the proposed design:

- The application will be hosted on AWS.
- Services should be independently deployable and scalable.
- The migration should be gradual, with a rollback option if issues occur.
- The application will require monitoring, health checks, and load testing before production cutover.
- The exact traffic pattern, performance targets, database requirements, and availability objectives will need to be confirmed before implementation.

3.Proposed Architecture Diagram



4. Architecture Decision Records (ADRs)

The following ADRs document the main architectural decisions for the proposed migration. Each decision considers the application requirements, available alternatives, and the trade-offs involved.

ADR-001: Incremental Migration Strategy

Status: Proposed

Context

The existing application is a monolith that needs to evolve into microservices. Replacing the entire application at once could introduce significant deployment risk and business disruption.

Decision

Adopt the **Strangler Fig pattern** to migrate business capabilities incrementally. The existing monolith will continue handling functionality that has not yet been migrated, while selected requests are routed to the new microservices.

Alternatives Considered

- **Big-bang migration:** Rejected because replacing the entire application at once increases cutover risk and makes troubleshooting and rollback more difficult.
- **Complete rewrite before release:** Rejected because it delays business value until the replacement is ready and increases the risk of discovering major issues late in the project.

Consequences

Incremental migration allows smaller releases, gradual validation, and easier rollback. However, the monolith and microservices must coexist temporarily, increasing routing and data consistency complexity.

ADR-002: Compute Platform — Amazon ECS with AWS Fargate

Status: Proposed

Context

The target architecture must support independently scalable microservices and a workload of 10,000 requests per second without introducing unnecessary operational complexity.

Decision

Use **Amazon ECS with AWS Fargate** to run containerised microservices. Each service will have its own task definition, deployment configuration, IAM permissions, and scaling policies where appropriate.

Alternatives Considered

- **Amazon ECS on EC2:** Not selected because the team would also need to manage the underlying instances, capacity, patching, and host lifecycle. It could be more cost-effective at sustained high utilisation.
- **Amazon EKS:** Not selected because Kubernetes introduces additional platform and operational complexity, and the brief does not identify a specific Kubernetes requirement.
- **Virtual machines running the monolith:** Not selected as the target approach because they do not provide the same service-level deployment and scaling model.

Consequences

Fargate reduces infrastructure management and supports independent service scaling. However, it may cost more than ECS on EC2 under sustained high utilisation. The 10,000 requests-per-second target must be validated through load testing rather than assumed from the choice of compute platform.

ADR-003: Communication Between Services

Status: Proposed

Context

Microservices need to communicate while avoiding unnecessary dependencies and maintaining reliable business operations. Some operations require an immediate response, while others can run in the background.

Decision

Use **synchronous HTTP APIs** for operations that require an immediate response, such as inventory reservation. Use asynchronous messaging through Amazon SQS or Amazon EventBridge for suitable background tasks and event-driven workflows.

Use ECS Service Connect or an equivalent service discovery mechanism so services communicate through stable service names rather than hardcoded task IP addresses.

Alternatives Considered

- **Synchronous communication for every operation:** Not selected because long-running downstream operations can increase response times and couple service availability.
- **Asynchronous messaging for every operation:** Not selected because some user-facing operations, such as confirming an inventory reservation, require an immediate result.
- **Hardcoded task IP addresses:** Rejected because task addresses can change as services scale or are redeployed.

Consequences

This approach supports immediate responses where necessary and reduces coupling for background processing. However, asynchronous workflows introduce eventual consistency and duplicate-message risks. Appropriate timeouts, bounded retries, idempotency, and atomic inventory updates will be required.

ADR-004: Database per Service and Data Migration

Status: Proposed

Context

During migration, the monolith and new microservices may operate simultaneously. Uncontrolled writes across systems could cause inconsistent inventory records, duplicate updates, or overselling. The target architecture must establish clear data ownership while allowing services to be migrated incrementally with minimal business disruption.

Decision

Adopt the **Database-per-Service pattern**. Each microservice will own its data and schema, and other services must access that data through the owning service's API or events rather than directly querying its database.

The initial target architecture will use **one Amazon Aurora Multi-AZ cluster containing separate logical databases for the Inventory, Order, and User services**. Each service will have dedicated database credentials and least-privilege permissions to access only its own database. The databases will share the underlying cluster infrastructure, while data ownership remains separated at the service level.

Migrate each service independently. For each migration, transfer the required data from the existing database, validate its accuracy and consistency, and then perform a controlled cutover. Maintain **one authoritative writer for each business capability at a time**. For example, the monolith will remain authoritative for inventory updates until the Inventory Service and its data are ready for cutover.

Rollback procedures must account for writes made after cutover. Separate Aurora clusters may be considered in the future if independent scaling, stronger infrastructure isolation, or availability requirements justify the additional cost and operational complexity.

Alternatives Considered

- **Shared database tables across all services:** Rejected because this creates tight coupling, weakens data ownership boundaries, and complicates independent schema changes.
- **Separate Aurora clusters for every service from the outset:** Not selected for the initial design because it increases infrastructure and operational complexity without a confirmed requirement for independent cluster-level isolation.
- **Uncoordinated dual writes:** Rejected because failures between writes can leave systems inconsistent.
- **Immediate database replacement:** Not selected because it increases cutover risk and reduces opportunities to validate data before switching traffic.

Consequences

Database-per-Service establishes clear data ownership and supports independent schema evolution. Using one Aurora cluster reduces initial infrastructure complexity, but the databases

still share cluster capacity and availability dependencies. Separate logical databases and credentials do not provide complete infrastructure-level isolation.

Cross-service workflows must use service APIs or events, which may introduce additional latency and eventual-consistency considerations. Controlled write ownership, data reconciliation, and tested rollback procedures remain essential. AWS Database Migration Service (AWS DMS) may be considered if the database engines and replication requirements support the proposed migration.

ADR-005: Network Architecture and Least Privilege

Status: Proposed

Context

The assessment requires network traffic to follow least-privilege principles. Application components should only be reachable through approved communication paths.

Decision

Deploy the architecture in a multi-Availability Zone VPC with:

- **Public subnets:** Application Load Balancer and required network egress infrastructure.
- **Private application subnets:** ECS Fargate tasks without public IP addresses.
- **Private data subnets:** Databases isolated from direct internet access.

Use security groups to allow only required source, destination, and port combinations. For example, application tasks should accept traffic from the load balancer's security group, while database access should be limited to authorised application security groups. Restrict outbound traffic where practical and use VPC endpoints for supported AWS services where appropriate.

Alternatives Considered

- **Running application tasks in public subnets:** Not selected because it increases direct exposure to the internet.
- **Allowing unrestricted inbound and outbound traffic:** Rejected because it violates least privilege and increases the potential impact of a compromised component.
- **Using network ACLs as the only security control:** Rejected because they do not replace the need for workload-specific security group rules and identity-based permissions.

Consequences

The design reduces unnecessary network exposure and provides clear communication boundaries. However, security group rules, routing, and outbound connectivity require careful configuration and testing. Network access controls must be complemented by IAM permissions and appropriate logging.

ADR-006: Encryption and PII Protection

Status: Proposed

Context

Regulatory requirements mandate encryption of personally identifiable information (PII) both at rest and in transit.

Decision

Enable encryption for relevant databases, backups, snapshots, and object storage. Use AWS Key Management Service (AWS KMS) where appropriate for encryption key management.

Use HTTPS/TLS for external traffic and TLS for internal service-to-service and application-to-database connections wherever supported. Apply least-privilege IAM and key policies, store credentials in AWS Secrets Manager, and avoid exposing PII in application logs.

Alternatives Considered

- **Encrypting only external traffic:** Rejected because traffic between internal services and databases could otherwise remain unencrypted.
- **Encrypting databases but not backups:** Rejected because backups and snapshots may also contain sensitive information.
- **Hardcoding credentials in application code:** Rejected because credentials could be exposed through source control or deployment artefacts.

Consequences

The decision protects sensitive data across storage and communication paths. However, certificates, keys, and secrets require ongoing management. The implementation must verify every relevant data flow, including internal connections, backups, and any storage services that contain PII.

ADR-007: Scalability, Capacity Planning, and Resilience

Status: Proposed

Context

The architecture must support 10,000 requests per second.

Decision

Use ECS service autoscaling with appropriate metrics, health checks, and multi-Availability Zone deployment. Validate capacity through end-to-end load testing, including application services, databases, and downstream dependencies.

Measure throughput, latency, error rates, CPU and memory utilisation, and database performance. Evaluate whether sufficient capacity remains after an Availability Zone failure if uninterrupted target throughput is a required business objective.

Alternatives Considered

- **Fixed task counts without scaling:** Not selected as the sole strategy because demand can change and fixed capacity may be insufficient during traffic spikes.
- **CPU-only autoscaling:** Not selected as the only scaling signal because bottlenecks may occur in databases, network calls, or request queues while CPU remains low.
- **Assuming autoscaling guarantees 10,000 requests per second:** Rejected because scaling cannot compensate for every bottleneck and additional capacity may not become available immediately.

Consequences

Testing and monitoring provide evidence for capacity decisions and help identify bottlenecks before production cutover. However, load testing requires time and resources, and scaling policies must be tuned using measured results. Database capacity, service quotas, and downstream dependencies must also be considered.

Summary

These seven ADRs establish the proposed migration approach, compute platform, communication patterns, data ownership, network controls, encryption strategy, and capacity planning method. The decisions aim to balance scalability, security, operational simplicity, and migration risk. They should be reviewed against the application's actual workload and business requirements before implementation.

5. Security and Compliance

The proposed architecture applies security controls at the network, application, and data layers to protect sensitive information and enforce least-privilege access.

5.1 Encryption and Data Protection

- **Encryption at rest:** Enable encryption for databases, backups, snapshots, and object storage containing PII. AWS KMS can be used to manage encryption keys.
- **Encryption in transit:** Use HTTPS/TLS for external traffic and TLS for internal service-to-service and application-to-database connections.
- **Secrets management:** Store database credentials, API keys, and other secrets in AWS Secrets Manager rather than in source code or configuration files.
- **PII protection:** Avoid logging sensitive personal information and restrict access to application data through IAM permissions.

5.2 Network Security and Least Privilege

Component	Security control
Application Load Balancer	Accept external application traffic only over HTTPS on port 443.
ECS Fargate services	Deploy in private subnets without public IP addresses. Allow inbound traffic only from authorised sources.
Database layer	Prevent direct internet access and allow connections only from authorised application services on required ports.
Service-to-service communication	Restrict traffic to required services, protocols, and ports using security groups and application-level controls.
AWS service access	Use IAM roles with least-privilege permissions and VPC endpoints where appropriate.

Security groups will define which components can communicate, while IAM policies will control access to AWS resources. These controls work together to reduce unnecessary access and limit the potential impact of a compromised component.

5.3 Monitoring and Compliance Validation

Enable appropriate logging and monitoring using Amazon CloudWatch and VPC Flow Logs. AWS CloudTrail can record relevant AWS API activity for auditing. Security testing should verify encryption settings, access permissions, network rules, and the absence of unintended public access before production cutover.

These controls provide a foundation for meeting the assessment's encryption and least-privilege requirements. Final compliance should be validated against the applicable regulatory standards and the organisation's specific policies.

6. Migration and Validation Plan

The migration will follow a phased approach to minimise business disruption, maintain data consistency, and validate the new architecture before fully switching traffic.

6.1 Migration Phases

Phase 1: Assessment and Preparation

Review the existing monolith, identify suitable business capabilities for extraction, map data dependencies, and prepare the AWS infrastructure and deployment pipelines.

Phase 2: Service Extraction

Extract a suitable business capability into an independent microservice and deploy it using Amazon ECS with AWS Fargate. Configure routing, service communication, security controls, and monitoring.

Phase 3: Data Migration and Validation

Transfer the required data to the new service and validate its accuracy and consistency. For inventory, maintain a single authoritative writer until the new service is ready for cutover. Plan the transfer of write ownership carefully to prevent conflicting updates.

Phase 4: Controlled Traffic Cutover

Gradually route the relevant requests from the monolith to the new service. Monitor application behaviour and data consistency throughout the transition. If critical issues occur, use the rollback procedure, ensuring that any new data changes are reconciled before returning write ownership to the monolith.

Phase 5: Monitoring and Optimisation

Once the service is stable, review its performance, resource utilisation, security controls, and scaling behaviour. Repeat the process for other business capabilities until the planned migration is complete.

6.2 Validation Plan

Before production cutover, perform the following checks:

- **Load testing:** Verify that the architecture can handle the target workload of 10,000 requests per second.

- **Performance testing:** Measure response times, error rates, and resource utilisation to identify bottlenecks.
- **Data validation:** Reconcile records between the monolith and the new service and verify that inventory updates remain consistent.
- **Security testing:** Confirm encryption at rest and in transit, least-privilege network rules, and appropriate IAM permissions.
- **Resilience testing:** Test service health checks, task replacement, scaling behaviour, and recovery from relevant component failures.
- **Rollback testing:** Verify that traffic can be redirected safely and that data changes made after cutover will not be lost.

The migration should proceed only when the relevant validation checks pass and the service meets the agreed acceptance criteria. Performance targets, recovery objectives, and cutover conditions should be confirmed before production deployment.

7. Conclusion

The proposed architecture provides a practical approach to migrating the existing monolithic application to microservices on AWS. By using Amazon ECS with AWS Fargate and an incremental migration strategy, the design supports independent service deployment and scaling while reducing migration risk.

Security is addressed through encryption at rest and in transit, private application and data layers, and least-privilege network and IAM controls. Load testing, monitoring, data validation, and rollback procedures will help verify the architecture before production cutover.

The final capacity requirements and production readiness must be confirmed through testing and validation against the target of 10,000 requests per second and the application's actual business and operational requirements.